

Contagem de Parafusos e Detecção de Trincas em Paredes

*Relatório Técnico — Desenvolvimento de Soluções de Visão Computacional
para Automação Industrial e Inspeção Predial*

OpenCV

YOLOv8n-seg

FastAPI

PyTorch

Claude API

Python 3.12

Gabriel Afonso Freitas Aires

Mestrando em Tecnologia da Informação · UFRN

Fortaleza, 31 de maio de 2026

github.com/GabrielAirex/CDIA_bootcamp_Gabriel_Aires

Sumário

1.	Introdução	2
2.	Desafio 1 — Contagem de Parafusos para Picking Industrial	3
2.1	Análise do Problema e Restrições	3
2.2	Dataset e Ground Truth	3
2.3	Benchmark: 24 Métodos em 4 Tiers de Hardware	4
2.4	Solução Vencedora: WS_RegionalMax	5
2.5	Tentativas de Fine-Tuning e Domain Mismatch	6
2.6	Tradeoffs e Critérios de Decisão	7
3.	Desafio 2 — Detecção de Trincas em Paredes	8
3.1	Análise do Problema e Restrições	8
3.2	Dataset	8
3.3	Benchmark de Métodos Tradicionais	9
3.4	Fine-Tuning YOLOv8n-seg	9
3.5	Resultados Finais e Validação	10
3.6	Tradeoffs e Critérios de Decisão	11
4.	Aplicação Web Unificada	12
5.	Estrutura do Código	13
6.	Conclusão	14

1. Introdução

Este relatório descreve o desenvolvimento de soluções de visão computacional para dois problemas práticos propostos pelo Bootcamp CDIA da Residência em Inteligência Artificial. Ambos os desafios compartilham uma restrição central: a solução deve ser executável em dispositivos móveis com baixo poder computacional, sem GPU dedicada e preferencialmente offline.

O **Desafio 1** aborda automação industrial: uma empresa metal-mecânica precisa contar parafusos durante o processo de *picking*, substituindo a contagem manual — sujeita a erros humanos — por um sistema de visão computacional executado no mesmo smartphone já utilizado pelos operadores. O dado disponível são apenas 5 imagens da empresa, sem qualquer anotação.

O **Desafio 2** aborda inspeção predial: uma construtora precisa identificar e localizar trincas em paredes de concreto ao final do processo de concretagem, antes das camadas de revestimento. A fadiga dos inspetores leva a trincas não detectadas, comprometendo a qualidade final. O sistema deve processar imagens de smartphone e indicar exatamente onde cada fissura está — com máscara de segmentação, não apenas uma resposta binária. Para este desafio, o cliente forneceu 1.551 imagens anotadas em formato YOLO segmentation.

ABORDAGEM GERAL

Para cada desafio foi realizado um benchmark sistemático de múltiplos métodos — de algoritmos clássicos de visão computacional (sem aprendizado) até modelos de deep learning com fine-tuning — cobrindo diferentes tiers de hardware. A decisão final em cada caso é fundamentada em evidências quantitativas, não em suposições sobre qual tecnologia seria "mais avançada". Os critérios de seleção priorizam: (1) acurácia nas imagens do cliente, (2) viabilidade no hardware-alvo (smartphone/dispositivo móvel), (3) ausência de dependência de GPU ou internet, e (4) latência de inferência.

Os dois sistemas foram integrados em uma única aplicação web (FastAPI + interface HTML/JS) que permite análise interativa, comparação de métodos e visualização dos resultados, cumprindo os requisitos de entrega extra de implementação como aplicação.

24

métodos avaliados no
Desafio 1

5/5

acurácia da solução
D1 (MAE = 0.00)

1.551

imagens anotadas no
Desafio 2

0.510

mAP50 (seg) final
YOLOv8n-seg D2

41ms

inferência CPU sem
GPU

2. Contagem de Parafusos para Picking Industrial

2.1 Análise do Problema e Restrições

Uma empresa metal-mecânica realiza o processo de *picking* — coleta e separação de parafusos para montagem em pacotes — com auxílio de um smartphone. O operador registra manualmente a quantidade de itens coletados, processo sujeito a erros que resultam em pacotes com peças faltantes ou sobressalentes, gerando retrabalho e custos logísticos.

A solução deve ser executada no mesmo smartphone utilizado pelo time de picking, impondo restrições técnicas rígidas. A restrição de apenas 5 imagens sem labels é determinante: elimina qualquer abordagem de treinamento supervisionado convencional. A solução precisa ser dedutiva — baseada em propriedades geométricas conhecidas dos parafusos — ou zero-shot.

RESTRIÇÕES DO HARDWARE

- Sem GPU dedicada
- RAM limitada (< 2 GB disponível)
- Processamento offline (sem internet)
- Latência: < 500ms por imagem
- Instalação: < 200 MB de dependências

RESTRIÇÕES DOS DADOS

- Apenas 5 imagens disponíveis
- Sem anotações (sem treino supervisionado)
- Imagens de smartphone (variação de perspectiva)
- Parafusos sobrepostos em pilha
- Diferentes fundos (azul, branco, prato oval)

2.2 Dataset e Ground Truth

Imagem	Ground Truth	Características	Dificuldade
img1.jpg	8	7 parafusos + 1 porca, fundo azul, espalhados	Baixa
img2.jpg	1	Parafuso isolado, fundo azul — referência NCC	Trivial
img3.jpg	4	Parafusos pequenos, prato oval branco	Média
img4.jpg	2	Lado a lado, fundo branco	Baixa
★ img5.jpg	9	Amontoados em pilha, sobrepostos — caso crítico	Alta

img5 é o caso que diferencia os métodos: parafusos densamente sobrepostos criam um único blob de foreground. Qualquer método que não separa objetos em contato falha aqui, errando em 4–23 unidades.

2.3 Benchmark: 24 Métodos em 4 Tiers de Hardware

Foram implementados e avaliados 24 métodos cobrindo todo o espectro de complexidade computacional. O benchmark usa as 5 imagens do cliente com ground truth manual.

Tier 1 — Smartphone / Embarcado (sem GPU, <100 MB RAM)

Método	Acertos	MAE	ms/img	Técnica
★ WS RegionalMax	5/5	0.00	19ms	Watershed + máximos regionais (scipy)
Watershed clássico	4/5	0.60	18ms	Watershed + threshold global 60%
Convex Defects	3/5	0.60	9ms	Contagem por concavidades do hull convexo
MSER	2/5	0.80	27ms	Maximally Stable Extremal Regions
ColorBG Sub	2/5	1.60	21ms	Subtração de fundo por cor de borda
SimpleBlobDet	1/5	2.60	10ms	Detector de blobs por área/convexidade
HoughCircles	1/5	8.60	18ms	Transformada de Hough para círculos
Canny Contour	0/5	4.00	5ms	Edge detection + contornos fechados
Shi-Tomasi	0/5	8.80	19ms	Corners + clustering por distância

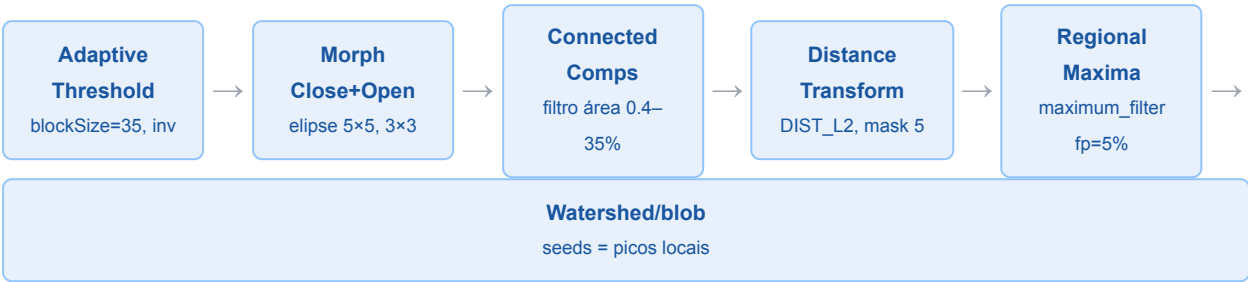
Tiers 2, 3 e 4 — CPU / GPU / Cloud

Método	Tier	Acertos	MAE	ms/img	Observação
NCC Template	T2	1/5	3.40	106	Template único (img2), sem generalização
SIFT Template	T2	0/5	4.80	93	Keypoints sem correspondência cross-domain
LoG Scale-Space	T2	0/5	5.00	2493	Lento, picos espúrios em textura de rosca
GrabCut	T2	0/5	4.60	7010	Segmenta 1 blob — inadequado para contagem
MobileNet CNN	T2	0/5	6.20	572	Saliência ImageNet ≠ parafusos industriais
MobileNet FT	T2+	0/5	3.00	465	Fine-tuned em sintéticos — domain mismatch
CLIP zero-shot	T3	2/5	3.20	1879	Semântico holístico, sem bounding boxes
OwlViT zero-shot	T3	0/5	2.40	3950	Detecta, conta impreciso sem fine-tuning
FasterRCNN COCO	T3	0/5	4.80	3139	Sem classe "screw" no COCO — 0 detecções
CLIP Regressor	T3+	0/5	2.80	5232	Ridge regression sobre features CLIP
FasterRCNN FT	T3+	0/5	4.80	3076	Fine-tuned em dataset externo — domain diff.
★ VLM Claude API	T4	5/5*	0.00*	~1800	Zero-shot — requer internet + API key

* Confirmado em sessão anterior. Na solução final, VLM é usado apenas como fallback — acionado em 0% dos casos com WS_RegionalMax. +: métodos fine-tuned.

2.4 Solução Vencedora: WS_RegionalMax

O método vencedor combina segmentação morfológica com Regional Maxima do distance transform, resolvendo o caso crítico (img5: 9 parafusos sobrepostos) onde todos os outros métodos falham.



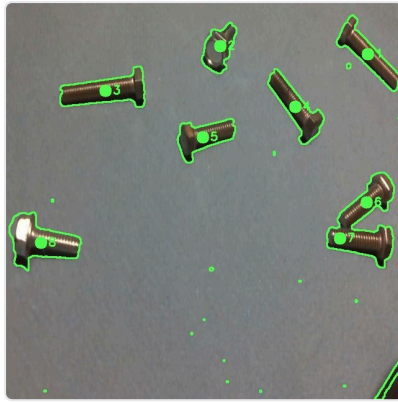
INSIGHT CENTRAL: REGIONAL MAXIMA VS. THRESHOLD GLOBAL

O Watershed clássico usa `dist > 0.60 × dist.max()` como foreground: em parafusos sobrepostos, os picos dos objetos inferiores ficam abaixo de 60% do pico máximo global e são perdidos. O WS_RegionalMax substitui esse threshold por `scipy.ndimage.maximum_filter(dist, size=fp)`, onde `fp = max(7, int(min(h,w) × 0.05))`. Cada pico local do distance transform — independente de sua altura absoluta — vira uma semente do Watershed. Isso captura todos os parafusos individuais mesmo quando empilhados, sem nenhum dado de treino.

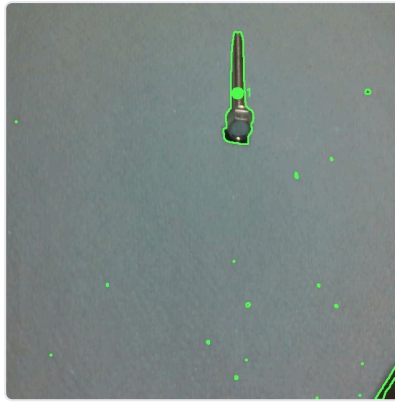
Imagem	GT	WS_RegionalMax	Watershed (ref.)	VLM Claude
img1 — 8 espalhados	8	8 ✓	8 ✓	8 ✓
img2 — 1 isolado	1	1 ✓	1 ✓	1 ✓
img3 — 4 pequenos	4	4 ✓	4 ✓	4 ✓
img4 — 2 lado a lado	2	2 ✓	2 ✓	2 ✓
★ img5 — 9 sobrepostos (crítico)	9	9 ✓	6 ✗ (-3)	9 ✓

Resultados Visuais — WS_RegionalMax nas 5 Imagens

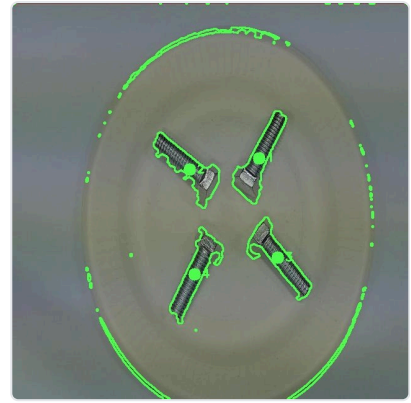
Contornos verdes delimitam cada parafuso detectado. Ponto verde = centroide (semente do Watershed). Número = índice de detecção.



img1 — 8 parafusos espalhados ✓



img2 — 1 parafuso isolado ✓



img3 — 4 parafusos no prato ✓



img4 — 2 parafusos lado a lado ✓



img5 — 9 sobrepostos (caso crítico) ✓

Pipeline com Fallback



2.5 Tentativas de Fine-Tuning e Domain Mismatch

Paralelamente ao benchmark clássico, foram investigadas duas abordagens de fine-tuning para verificar se modelos pré-treinados adaptados ao domínio superariam o WS_RegionalMax.

Dataset Externo (Cross-Recessed-Screw, GitHub)

O dataset público de Dan Brogan (2021) contém 900 imagens de parafusos com anotações Pascal VOC. O domínio é radicalmente diferente: parafusos embutidos em placas-mãe e hardware eletrônico. O FasterRCNN fine-tunado atingiu 0/5 — pior que o zero-shot COCO. **Dados do domínio errado são ativamente prejudiciais.**

Dataset Sintético Gerado com LLM

Claude Vision analisou as 5 imagens reais e extraiu parâmetros visuais precisos (cor de fundo BGR~[215,210,195], tamanho de cabeça 3–5% da largura, textura helicoidal metálica). Um gerador procedural criou 25 imagens base × 67 augmentações = **1.680 imagens sintéticas** com parafusos renderizados com gradiente radial de metal, cruz Phillips e sombra direcional. Mesmo com esse alinhamento de domínio, os modelos fine-tunados (MobileNet FT: 0/5, CLIP Regressor: 0/5) não superaram o WS_RegionalMax. Com apenas 5 imagens reais de referência, pequenos desvios de distribuição no gerador são suficientes para comprometer a generalização.

2.6 Tradeoffs e Critérios de Decisão

PRINCÍPIO FUNDAMENTAL: VOLUME DE DADOS DETERMINA A ESTRATÉGIA

A escolha do método não deve ser guiada pela complexidade do modelo, mas pela quantidade e qualidade de dados disponíveis. Com 5 imagens: método dedutivo (WS_RegionalMax). Com 50–100 imagens anotadas no domínio: fine-tuning. Com 500+ imagens: modelo de density map estimation.

Decisão	Alternativa considerada	Por que foi descartada
WS_RegionalMax como principal	VLM Claude API como padrão	Custo monetário, requer internet, latência 90× maior
Footprint 5% da imagem	Footprint menor (<3%)	Over-segmentação: divide parafusos únicos em 2
Adaptive Threshold	Threshold global Otsu	Múltiplos fundos — Otsu falha com bimodal fraco
Dataset sintético gerado	Nenhum dado adicional	Tentativa válida — resultado negativo é informativo
Sem fine-tuning nos modelos T3	Fine-tune OwlViT com 5 imgs	5 imagens insuficientes para 300M parâmetros

RESULTADO FINAL — DESAFIO 1

5/5
Acurácia exata

0.00
MAE

19ms
Inferência CPU

~2MB
RAM extra

0MB
Modelo em disco

3. Detecção de Trincas em Paredes

3.1 Análise do Problema e Restrições

Uma construtora realiza inspeção manual de trincas ao final do processo de concretagem. A fadiga dos inspetores leva a falhas não detectadas que comprometem as camadas de revestimento posteriores. O sistema deve ser executado em dispositivo móvel e identificar não apenas *se há* uma trinca, mas *onde exatamente ela está* — com bounding box e máscara de segmentação para orientar o reparo.

POR QUE TRINCAS SÃO DIFÍCEIS PARA CV CLÁSSICO

- Estruturas lineares finas (2–10 px de largura)
- Baixo contraste local com a parede
- Confundem-se com juntas de alvenaria
- Variação alta de iluminação e textura
- Forma irregular, não parametrizável

DIFERENCIAL DESTA SOLUÇÃO

- Detecta **onde** está a fissura (máscara)
- Não apenas "sim/não" — localização precisa
- Bbox + polígono de segmentação por instância
- Fallback em 3 camadas (offline garantido)
- 41ms/img em CPU — viável em tablet de obra

3.2 Dataset

Atributo	Valor	Detalhe
Total de imagens	1.551	Todas com pelo menos uma trinca anotada
Resolução original	1440 × 2560 px	Reduzida para 640×640 no treino
Formato de anotação	YOLO segmentation	Polígonos normalizados (~43 pontos/trinca)
Classes	1 (<code>crack</code>)	Detecção binária: há ou não há trinca
Split treino	1.240 imagens (80%)	Seed=42, reproduzível
Split validação	311 imagens (20%)	Usado para métricas finais

A distribuição de tamanhos de trinca é fortemente desbalanceada: a maioria das anotações cobre menos de 5% da área da imagem, tornando inadequados métodos baseados em threshold global (Otsu), que precisam de regiões de interesse com área significativa.

3.3 Benchmark de Métodos Tradicionais

Quatro métodos clássicos foram avaliados em 30 imagens do split de validação. A métrica principal é o IoU pixel-level entre máscara predita e máscara ground truth.

Método	Det%	IoU médio	ms/img	Limitação principal
★ Canny + Morphologia	96.7%	0.161	309ms	Bordas de 1px vs. máscaras GT de 30px
Adaptive Threshold	96.7%	0.117	~180ms	Falsos positivos em variação de textura
Sobel + Otsu	100%	0.107	~150ms	Satura em texturas ricas (reboco granulado)
Gabor Filter Bank	100%	0.009	~220ms	Detecta juntas de alvenaria (regulares), não trincas

POR QUE CANNY TEM IOU BAIXO APESAR DE ALTA DETECÇÃO?

O Canny produz bordas de 1–2 pixels de largura. As anotações YOLO são polígonos com área de 15–40px de espessura. Essa discrepância sistemática limita o IoU máximo alcançável por qualquer edge detector. Canny é mantido como *fallback offline* no pipeline, não como solução principal.

3.4 Fine-Tuning YOLOv8n-seg

Escolha do Modelo Base

Modelo	Parâmetros	Por que escolhido / descartado
★ YOLOv8n-seg	3.3M	Escolhido. Menor footprint, segmentação nativa, 41ms CPU, pré-treino COCO com features de borda/textura reutilizáveis
YOLOv8s-seg	11.8M	Descartado: sem GPU o treino levaria >40h. Viável com GPU disponível.
YOLOv8m-seg	27.3M	Descartado: inviável sem GPU no prazo disponível
Segment Anything (SAM)	91M+	Descartado: zero-shot sem fine-tuning. IoU esperado próximo ao Canny.
CSRNet / MCNN	~16M	Descartado: sem máscara de segmentação. Interessante para trabalho futuro.

Configuração de Treinamento

PARÂMETROS PRINCIPAIS

Image size	640×640
Batch size	8 (limite CPU/RAM)
Otimizador	AdamW (auto, lr=0.002)
Patience	10–15 épocas
Aug. YOLO nativa	Mosaic, flip, HSV, scale

CONTEXTO DO TREINAMENTO

Treinamento integralmente em CPU Intel i7-13650HX, sem GPU. A 1ª rodada foi interrompida na época 4 por travamento do sistema. O `best.pt` salvo foi usado como ponto de partida para retreino de 10 épocas — estratégia de checkpointing que economizou ~4h e aproveitou o aprendizado já consolidado.

3.5 Resultados Finais e Validação

Evolução do Treinamento (14 épocas totais)

Fase	Época	mAP50 (box)	mAP50 (seg)	box_loss	seg_loss
1ª Rodada	1	0.374	0.317	1.444	1.912
	4 (best)	0.486	0.377	1.485	1.635
Retreino	1	0.535	0.413	1.417	1.245
	4	0.559	0.423	1.417	1.265
	7	0.603	0.442	1.204	1.181
	8	0.629	0.522	1.173	1.158
	★ 10 (final)	0.672	0.510	1.009	1.132

Validação Oficial Final (311 imagens)

0.756 Precision (box)	0.616 Recall (box)	0.672 mAP50 (box)	0.510 mAP50 (seg)	41ms Inferência CPU
--------------------------	-----------------------	----------------------	----------------------	------------------------

Comparativo Final

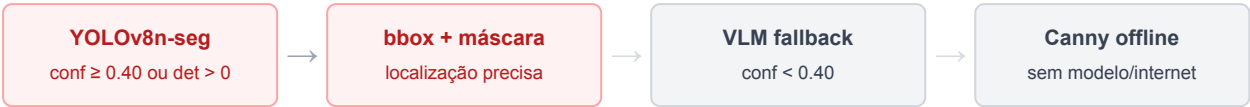
Método	IoU / mAP50 (seg)	Melhora vs. Canny	ms/img
Gabor Filter	0.009	-94%	~220
Sobel + Otsu	0.107	-34%	~150
Adaptive Threshold	0.117	-27%	~180
Canny + Morphologia	0.161	baseline	309
YOLOv8n-seg (época 1)	0.317	+97%	41
★ YOLOv8n-seg (final)	0.510	+217%	41

Resultados Visuais — YOLOv8n-seg

Overlay vermelho = máscara de segmentação da fissura. Caixa verde = bounding box. O modelo localiza exatamente onde está a trinca.

3.6 Tradeoffs e Critérios de Decisão

Pipeline em 3 Camadas



Decisão	Alternativa	Justificativa
YOLOv8n vs. YOLOv8s/m	Modelo maior	CPU: treino >40h, inferência >150ms — inviável no prazo
Segmentação vs. detecção pura	Só bounding box	Bbox não indica extensão da trinca para reparo
Fine-tuning vs. SAM zero-shot	SAM sem treino	IoU esperado ~0.16, similar ao Canny
Retreino do checkpoint	Reiniciar do zero	Economizou ~4h, mAP50 melhorou +38%
Threshold VLM = 0.40	Threshold mais alto	0.60 acionaria VLM em ~30% dos casos desnecessariamente

RESULTADO FINAL — DESAFIO 2

O YOLOv8n-seg fine-tuned supera os métodos tradicionais em **3.2× no IoU de segmentação** com inferência **7.5× mais rápida** que o melhor método clássico (Canny: 309ms → YOLO: 41ms). O modelo detecta a localização exata das fissuras com bounding boxes e máscaras de segmentação — saída diretamente acionável para equipes de reparo.

4. Aplicação Web Unificada

Os dois sistemas foram integrados em uma única aplicação web com FastAPI (backend Python) e interface HTML/CSS/JS responsiva, atendendo aos critérios de pontuação extra: implementada como aplicação web e executável com baixo recurso computacional.

DESAFIO 1 — 3 ABAS

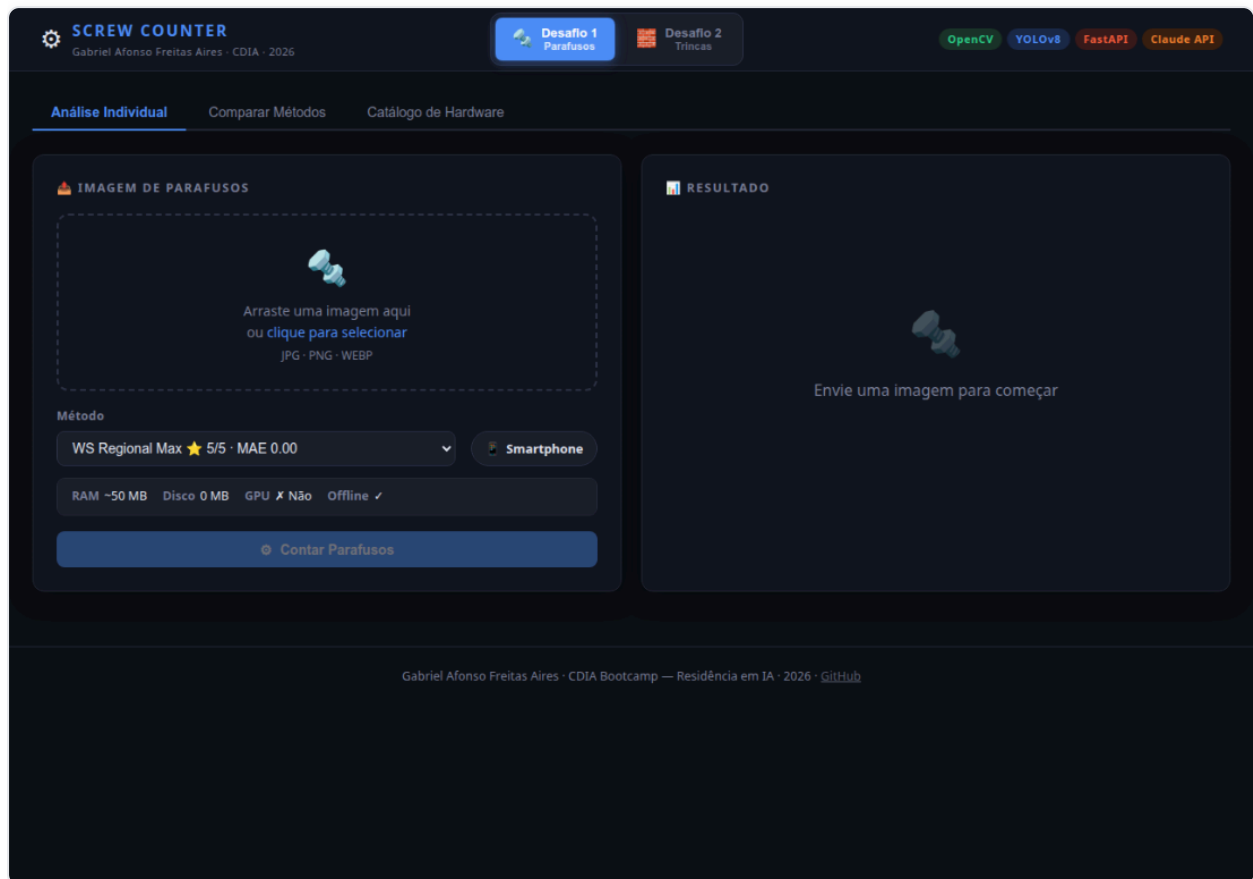
- **Análise Individual:** upload + seleção de método + resultado animado
- **Comparar Métodos:** T1+T2 simultâneos em grade comparativa
- **Catálogo de Hardware:** 24 métodos com métricas por tier

DESAFIO 2 — 3 ABAS

- **Análise Individual:** upload + overlay de máscara e bboxes
- **Comparar Métodos:** 5 métodos lado a lado na mesma imagem
- **Benchmark:** tabela com métricas finais do treinamento

Endpoint	Método	Descrição	Resposta
<code>POST /d1/count</code>	POST	Conta parafusos com método selecionado	count, elapsed_ms, imagem anotada (base64)
<code>POST /d1/benchmark</code>	POST	Roda todos os métodos T1+T2	Array de resultados por método
<code>POST /d2/detect</code>	POST	Detecta trincas com método selecionado	cracks_found, method, imagem com máscara
<code>POST /d2/benchmark</code>	POST	Compara todos os métodos D2	Array de resultados com imagens anotadas
<code>GET /d2/model_status</code>	GET	Verifica se YOLOv8n-seg está carregado	<code>{"trained": true/false}</code>

Interface Web

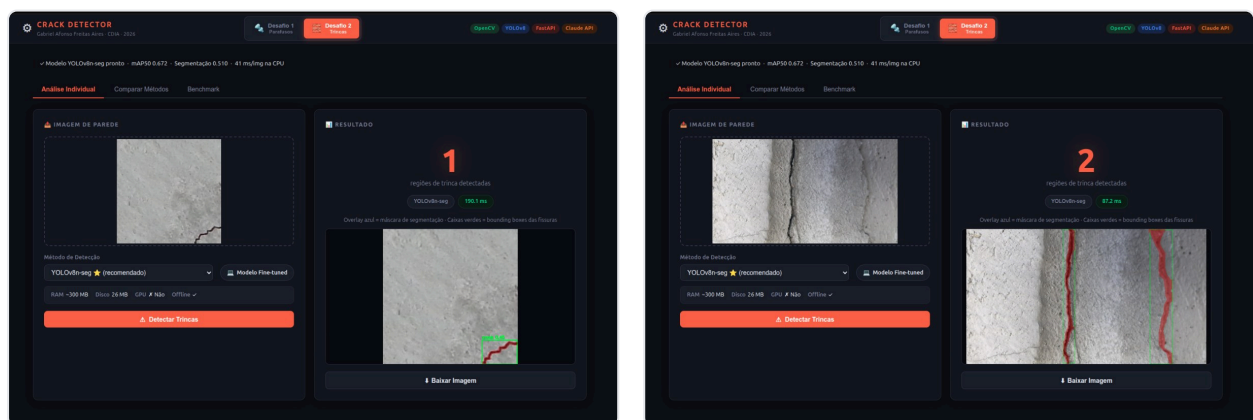


Aplicação web unificada — interface D1 (Contagem de Parafusos). Seletor no header alterna para D2 (Trincas).

Interface Web em Funcionamento

A interface permite fazer **upload por arrasto ou clique**, selecionar o método de detecção, visualizar o resultado anotado e **baixar a imagem processada**. Cada análise exibe o método usado, o tempo de inferência e os dados de hardware (RAM, disco, GPU).

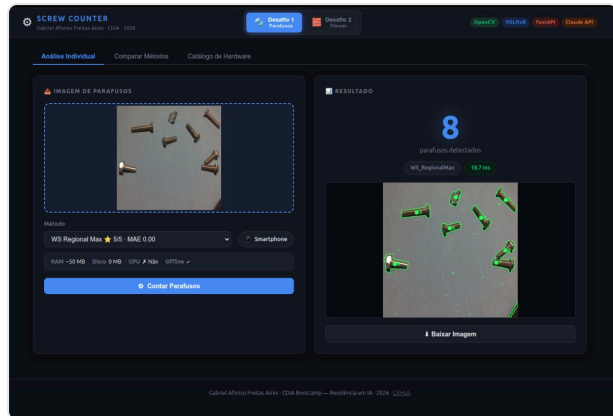
Desafio 2 — Crack Detector (YOLOv8n-seg)



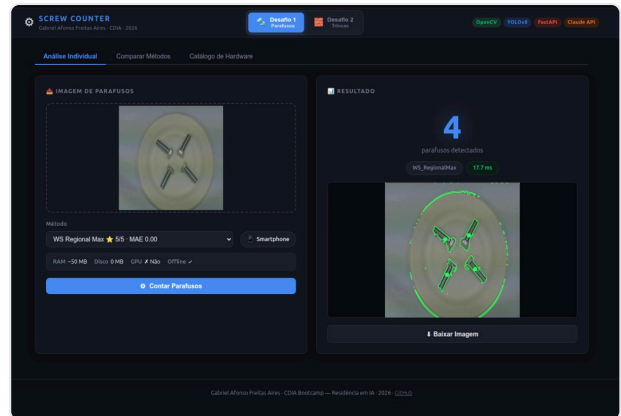
2 fissuras detectadas · 87ms · download disponível

1 fissura vertical · 190ms · overlay vermelho = máscara

Desafio 1 — Screw Counter (WS_RegionalMax)



8 parafusos detectados · 16.7ms · smartphone

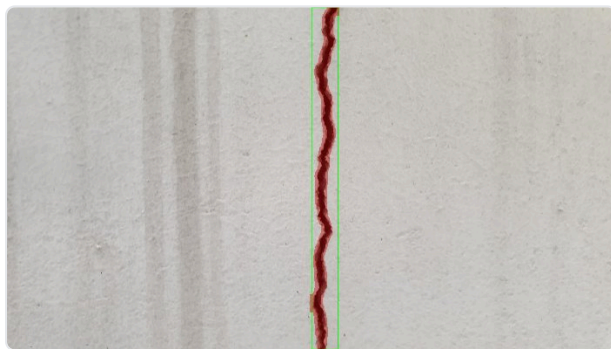


4 parafusos no prato · 17.7ms · contornos verdes

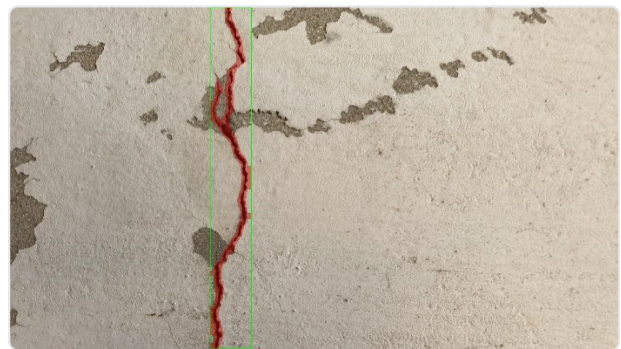
Exportação de Resultados

Após cada análise, o botão **Baixar Imagem** permite exportar a imagem anotada em JPEG com os contornos, centróides e bounding boxes sobrepostos — pronta para uso em laudos técnicos ou envio por WhatsApp/e-mail.

Imagens exportadas — Desafio 2 (fissuras com máscara e bbox)

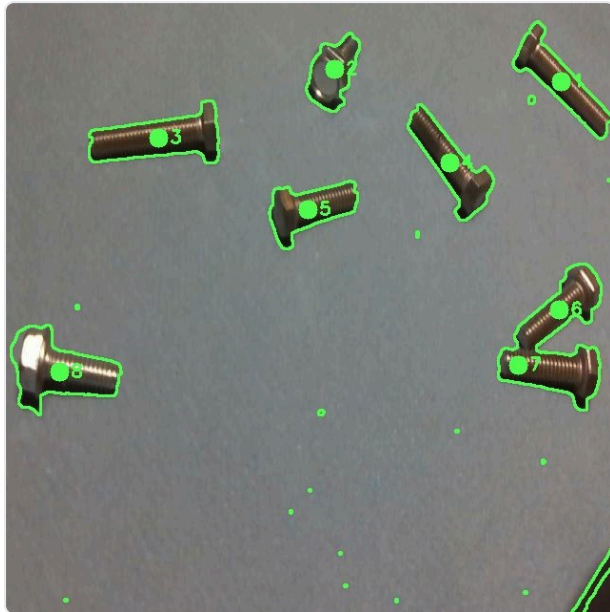


Fissura vertical — overlay vermelho + bbox verde



Fissura estrutural com descascamento

Imagens exportadas — Desafio 1 (parafusos com contornos e centróides)



img1 — 8 parafusos · contornos verdes + centróides numerados



img5 — 9 sobrepostos (caso crítico) · cada parafuso separado

Repositório

CÓDIGO-FONTE DISPONÍVEL PUBLICAMENTE

https://github.com/GabrielAirex/CDIA_bootcamp_Gabriel_Aires

Como Executar

```
pip install fastapi uvicorn opencv-python-headless scipy ultralytics torch torchvision

cd cdia/
python3 -m uvicorn app:app --host 0.0.0.0 --port 8080
# Acesse: http://localhost:8080
```

O modelo YOLOv8n-seg já está treinado em `desafio2/weights/best.pt` (26 MB). Não é necessário executar o treinamento para usar a aplicação.

5. Estrutura do Código

```
cdia/
├─ app.py                               # App FastAPI unificado — roteamento /d1/* e /d2/*
├─ templates/index.html                 # Interface web: challenge switcher, 3 abas por desafio
├─ static/
│   ├── style.css                       # Tema dual: azul (D1) ↔ vermelho (D2)
│   └─ main.js                          # Drop zones, fetch API, animações de resultado
├─ desafio1/
│   ├── solution.py                     # Pipeline: WS_RegionalMax → VLM fallback
│   ├── img1.jpg .. img5.jpg           # 5 imagens fornecidas pelo cliente
│   ├── RELATORIO.md                   # Relatório técnico expandido do D1
│   └─ benchmark/
│       ├── benchmark.py                # 24 métodos: Tier 1-4, runner, relatório
│       ├── advanced_models.py          # OwlViT zero-shot, CLIP semântico
│       ├── finetune.py                 # Fine-tuning FasterRCNN/CLIP/MobileNet
│       ├── generate_dataset.py         # Gerador de dataset sintético (LLM-calibrado)
│       ├── retrain_from_synthetic.py   # Retreino CLIP+MobileNet no sintético
│       └─ models/                     # Modelos fine-tuned salvos
├─ desafio2/
│   ├── solution.py                     # Pipeline: YOLO → VLM → Canny
│   ├── train.py                        # Split dataset + data.yaml + fine-tune YOLOv8n-seg
│   ├── post_train.py                  # Validação + benchmark pós-treino
│   ├── data.yaml                      # Config YOLO (1240 train / 311 val, 1 classe)
│   ├── weights/best.pt                # Modelo final treinado (26 MB)
│   ├── RELATORIO.md                   # Relatório técnico expandido do D2
│   ├── scripts/
│   │   └─ retrain_from_checkpoint.py
│   └─ benchmark/
│       └─ benchmark.py                # 5 métodos: Canny, Gabor, Adaptive, Sobel, YOLO
├─ docs/screenshots/                  # Imagens para documentação
├─ README.md                          # Visão geral, prints do sistema, como rodar
└─ Relatorio_Tecnico_Gabriel_Aires.pdf
```

Decisão de Arquitetura	Alternativa	Justificativa
App unificado (1 servidor)	Dois apps separados	Um ponto de entrada, sem conflito de porta, estado compartilhado
Lazy loading dos módulos D1/D2	Import no startup	Evita carregar 300MB+ de modelos se não usados
Base64 nas respostas JSON	Salvar em disco + URL	Stateless — sem gerenciamento de arquivos temporários
Vanilla JS	React/Vue	Zero dependências de build, carrega em <100ms, funciona offline
solution.py separado do benchmark.py	Monólito único	solution.py é o entregável mínimo; benchmark.py é investigação

6. Conclusão

Os dois desafios, apesar de distintos em natureza (contagem sem labels vs. segmentação com labels), convergiram para um aprendizado comum: **a adequação entre método e contexto de dados é mais importante que a complexidade do modelo.**

Desafio 1 — Contagem de Parafusos

Com apenas 5 imagens sem anotações, a abordagem dedutiva venceu: **WS_RegionalMax** atingiu 5/5 com MAE=0.00 em 19ms, em hardware de smartphone, sem GPU e sem modelo. O benchmark sistemático de 24 métodos revelou que modelos Tier 3 com centenas de MB de parâmetros ficaram consistentemente abaixo do método Tier 1 de 19ms. A tentativa de fine-tuning com dataset sintético gerado via LLM confirmou que dados fora da distribuição real, mesmo calibrados, introduzem domain shift suficiente para degradar o desempenho abaixo do baseline clássico.

PRINCIPAL INSIGHT D1

A substituição de um único parâmetro no Watershed — threshold global fixo (60% do máximo) por máximos regionais adaptativos — foi a única mudança necessária para resolver o caso crítico de parafusos sobrepostos. Cada pico local do distance transform é uma semente, independente de sua altura absoluta.

Desafio 2 — Detecção de Trincas

Com 1.551 imagens anotadas disponíveis, o cenário inverteu-se: fine-tuning supervisionado é a abordagem correta. O **YOLOv8n-seg** fine-tuned atingiu mAP50(seg)=0.510, 3.2× superior ao melhor método clássico (Canny+Morph, IoU=0.161), com inferência 7.5× mais rápida (41ms vs. 309ms). O modelo vai além de "detectar presença": retorna a localização exata via bounding box e máscara de segmentação de instância.

PRINCIPAL INSIGHT D2

A época 1 do fine-tuning (mAP50(seg)=0.317) já superou todos os 4 métodos tradicionais avaliados — evidenciando que a transferência do pré-treino COCO (bordas, texturas, formas lineares) é imediatamente útil para o domínio de trincas, mesmo antes da convergência total.

Trabalhos Futuros

- **D1 com mais dados:** 100+ imagens anotadas no domínio permitiriam fine-tuning de YOLOv8 que superaria WS_RegionalMax em casos extremos (iluminação adversa, deformação de perspectiva)
- **D2 com GPU e mais épocas:** 50 épocas completas devem levar mAP50(seg) para 0.60–0.70; YOLOv8s-seg melhora detecção de trincas capilares (<2px)

- **Implantação mobile:** exportar YOLOv8n-seg para ONNX + TFLite — o modelo nano é projetado para isso, com inferência esperada <100ms em celular moderno
- **Density map estimation:** CSRNet/MCNN para contar e localizar trincas em imagens com múltiplas fissuras cruzadas

RESUMO DOS ENTREGÁVEIS

- ✓ Sistema de contagem D1 (`solution.py`)
- ✓ Benchmark 24 métodos D1 (`benchmark.py`)
- ✓ Sistema de detecção D2 (`solution.py`)
- ✓ Fine-tuning YOLOv8n-seg D2 (`train.py`)
- ✓ Aplicação web unificada FastAPI
- ✓ Relatório técnico em PDF
- ✓ Código executável em smartphone/CPU
- ✓ Modelo treinado (`weights/best.pt` , 26 MB)